PaseQR — Boletos digitales con QR en la nube

Proyecto Integrador · Desarrollo en la Nube · Otoño 2026 · Prof. Marcela Rosales, ITESO · **Fase 2: Entrega de avances**

Equipo: David Valdez · Vittorio Catino · Juan Pablo Gutiérrez
Repositorio: github.com/davidvaldezm/proyecto-desarrollo-en-la-nube | **Video:** https://youtu.be/R3J9fsPNys8
App en AWS: paseqr-dev-web-901501251693.s3-website-us-east-1.amazonaws.com | **API:** `https://p3vllywmbf.execute-api.us-east-1.amazonaws.com`

1. Problema y aplicación (actualizado)

Los eventos pequeños y medianos (conciertos universitarios, conferencias, torneos) suelen venderse con hojas de cálculo y listas impresas. Eso provoca **sobreventa**, **boletos falsificados o compartidos** por captura de pantalla, filas lentas y ningún dato de asistencia. **PaseQR** permite al organizador crear un evento con cupo y un PIN para su staff. El asistente compra y recibe un **PDF con un QR firmado**; el staff valida la entrada desde el celular y el organizador recibe recordatorios y un reporte de asistencia. La nube es la solución adecuada porque la demanda llega **en picos** (apertura de preventa y hora de entrada) y casi es nula el resto del tiempo. Una arquitectura serverless escala sola, cuesta cero sin tráfico y garantiza atomicidad con DynamoDB aunque cientos de personas compren al mismo tiempo. El pago se simula.

2. Flujos end-to-end (los tres funcionan en AWS)

Flujo	Entrada → lógica	Resultado observable y cómo lo probamos
1. Compra y emisión `propio de la nube` David	`POST /compras` valida nombre, correo, cantidad (1–4) y `compra_id` (UUID). Una **transacción de DynamoDB** aparta los lugares solo si hay cupo y crea los boletos *PENDIENTE*. Se encola un mensaje por boleto en **SQS**. La Lambda *generar-boleto* firma el QR, genera el PDF (reportlab), lo guarda en **S3**, marca el boleto *EMITIDO* y avisa por **SNS**.	Boleto PDF descargable en "Mis boletos" (`GET /boletos`, URL prefirmada) y correo. **Prueba en AWS:** 50 compras simultáneas contra cupo de 10 → **10 × 202, 40 × 409, vendidos = 10**, sin sobreventa. Los PDFs se emitieron en segundos.
2. Check-in con QR Vittorio	`POST /checkin` con evento, PIN y QR: (1) verifica la firma HMAC-SHA256 en tiempo constante; (2) verifica que el QR sea del evento; (3) **update condicional** EMITIDO → USADO y suma un asistente.	Pantalla verde o roja con el motivo y el contador de asistentes. **Prueba en AWS** con el QR leído del PDF real: válido → *VALIDO*, segundo escaneo → *YA_USADO*, un carácter alterado → *INVALIDO*, PIN incorrecto → 403.
3. Recordatorios y reporte Juan Pablo	Una regla de **EventBridge** ejecuta cada hora la Lambda *recordatorios*: avisa a los asistentes de eventos que empiezan en las próximas 24 h y genera el reporte de los eventos que ya terminaron. Una marca condicional evita envíos duplicados.	Correo de recordatorio y **CSV en S3** (vendidos, asistentes, tasa de asistencia, hora pico, ingresos), con enlace al organizador. Para el demo se puede invocar a mano con `{"accion":"reporte","evento_id":…}`.

3. Servicios de AWS y cambios respecto a la Fase 1

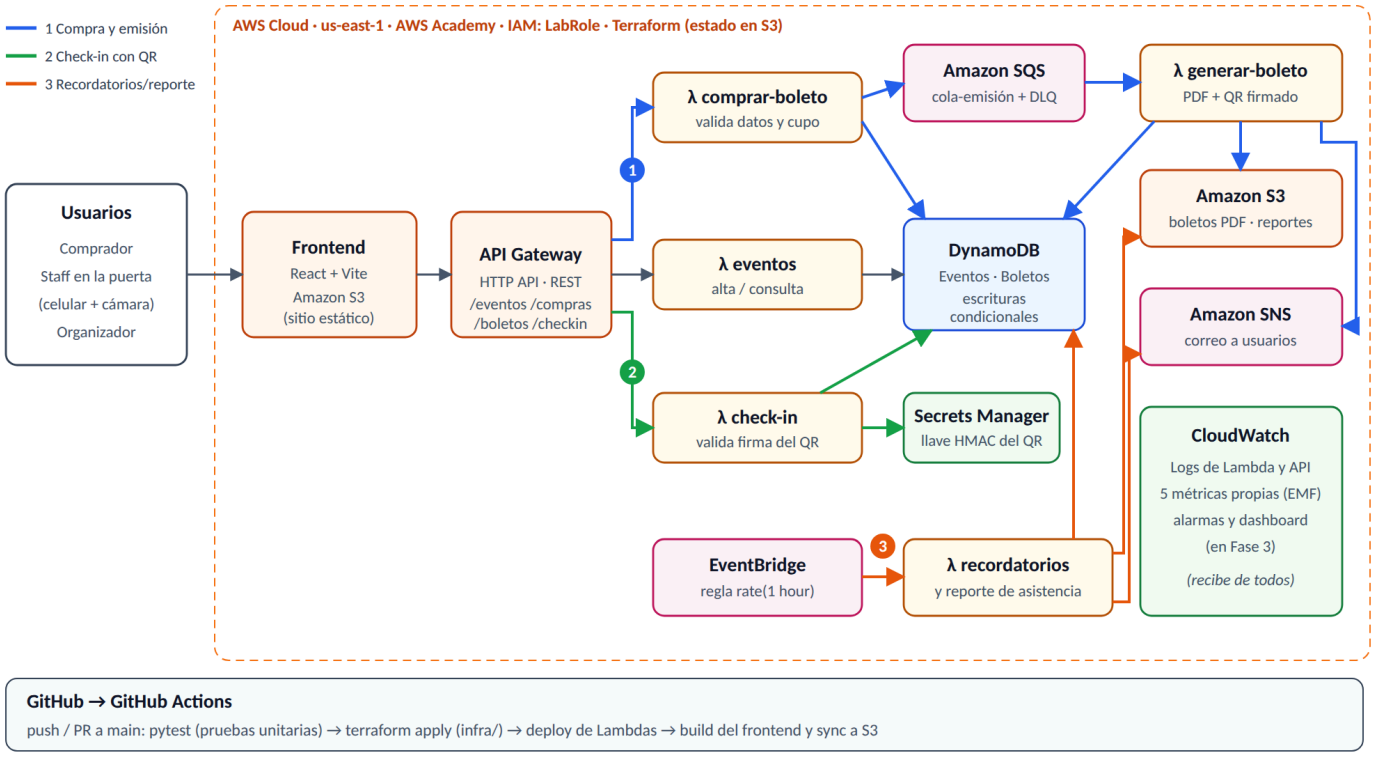
Servicio	Uso	Por qué / cambio
Lambda (5)	eventos, comprar-boleto, generar-boleto, check-in, recordatorios. Python 3.12 y un solo paquete.	Escala con los picos y no cuesta sin tráfico.
API Gateway (HTTP)	6 rutas, CORS y *throttling* (100 rps, ráfaga de 200).	Protege el backend durante la preventa.
DynamoDB	Tablas Eventos y Boletos (GSI *por_evento* y *por_email*), en modo on-demand.	Transacciones y escrituras condicionales atómicas: sin sobreventa ni doble entrada.
S3	Frontend estático, PDFs y reportes (privados, con URL prefirmada).	Almacenamiento durable; descarga sin exponer el bucket.
SQS + DLQ	Cola de emisión; 3 intentos antes de ir a la DLQ.	Desacopla la compra del PDF y absorbe las ráfagas.
SNS	Correos con filtro por destinatario (atributo `email`).	Notificaciones administradas.
Secrets Manager	Llave HMAC de los QR, cacheada por contenedor.	La llave no vive en el código ni en el repositorio.

CloudWatch	Logs y métricas propias con EMF: BoletosVendidos, BoletosEmitidos, CheckInsValidos, CheckInsRechazados, ComprasRechazadasPorCupo.	EMF no requiere permisos extra. Las alarmas y el dashboard quedan para la Fase 3.
EventBridge	Regla `rate(1 hour)` que dispara el Flujo 3.	**Cambio:** se usa una regla en lugar de *Scheduler*, porque Scheduler requiere un rol de IAM y AWS Academy no permite crearlo.

4. Decisiones técnicas y su justificación

- **Transacción de DynamoDB e idempotencia (Tarea 2, David).** DynamoDB no permite aritmética en las condiciones, así que la regla "vendidos + n ≤ cupo" se expresa como `vendidos ≤ cupo − n`. Los boletos tienen IDs deterministas (`compra_id#1…n`) con `attribute_not_exists`, de modo que un reintento devuelve los mismos boletos. **Hallazgo en la nube:** la primera prueba de carga dio 14 errores 500 por *TransactionConflict* (transacciones simultáneas sobre el mismo evento). La corregimos con reintentos con espera exponencial y jitter; si el conflicto persiste se responde 503.
- **QR firmado con HMAC más update condicional (Tarea 2, Vittorio).** La firma detecta falsificaciones sin consultar la base. La condición `estado = EMITIDO` resuelve las copias y los escaneos simultáneos.
- **Emisión asíncrona e idempotente.** SQS entrega "al menos una vez", así que el paso PENDIENTE → EMITIDO lleva condición y la llave del PDF en S3 es fija.
- **Restricciones de AWS Academy.** (a) Las Lambdas usan `LabRole`. (b) La política de la organización niega `s3:GetBucketObjectLockConfiguration`, que el recurso `aws_s3_bucket` de Terraform consulta, así que los buckets se crean con la CLI en `deploy.sh` y Terraform los referencia como *data* y los configura. (c) **CloudFront no está permitido**, así que el sitio es HTTP: la cámara en vivo no está disponible y el check-in usa una **foto del QR** (o captura manual), que funciona en HTTP.
- **Infraestructura como código y CI/CD.** Terraform guarda su estado en S3 con *lockfile*. GitHub Actions corre pytest, el build y `terraform validate` en cada rama, y despliega con `scripts/deploy.sh` en cada push a `main`. Si el deploy falla, publica las últimas líneas del log como anotación del run.
- **Pruebas.** 35 pruebas unitarias con pytest y moto: sobreventa, carrera con lectura desactualizada, reintento, conflicto, PDF y QR, firma alterada o forjada, otro evento, PIN incorrecto, recordatorio único y reporte. moto no es seguro con hilos, por eso la concurrencia real se prueba con `scripts/prueba_carga.py` contra AWS.

5. Diagrama de arquitectura



Toda la aplicación corre en AWS (us-east-1). Los números indican los tres flujos end-to-end.

6. Qué corre en la nube y qué en local

Componente	Dónde	Detalle
Frontend (React + Vite)	**Nube**	Bucket S3 con sitio estático, publicado por el pipeline.
API y las 5 Lambdas	**Nube**	API Gateway + Lambda, desplegadas con Terraform.
Datos, archivos y mensajería	**Nube**	DynamoDB, S3, SQS + DLQ, SNS y Secrets Manager.
Tarea programada	**Nube**	Regla de EventBridge cada hora.
Pipeline CI/CD	**Nube**	GitHub Actions (pruebas + deploy en cada push a `main`).
Pruebas unitarias	Local / CI	pytest + moto (AWS simulado); también corren en CI.

Avance: aproximadamente el **90 %** de la aplicación funciona en la nube. Ningún componente de la app depende de la máquina local.

7. Roles y lo que falta para la Fase 3

Integrante	Rol	Pendiente
David Valdez	Flujo 1 + Frontend	Pulir la interfaz, prueba de carga más grande (500 compras) y su reporte.
Vittorio Catino	Flujo 2 + CI/CD	Flujo de Pull Requests con revisión obligatoria y ambiente `prod` separado de `dev`.
Juan Pablo Gutiérrez	Flujo 3 + Monitoreo	**2 alarmas** (5xx de la API > 5 % y mensajes en la DLQ > 0) y **dashboard** de CloudWatch en Terraform.

También queda pendiente agregar autenticación para el organizador (hoy los boletos se consultan solo con el correo) y evaluar un plan B si no hay internet en la puerta del evento.